Here is the comprehensive technical specification with all mathematical symbols converted to simple text for easy reading and rendering.

Unified Technical Specification: Reversible Audio-Video Steganography System

Part 1: System Assumptions & Temporal Synchronization

This section defines the fundamental constraints ensuring deterministic alignment between audio and video modalities without requiring cross-chunk context.

1.1 Global Constants & Sync Map

The system relies on a Global Sync Map defined by integer time intervals. For any integer **k**, a "Chunk" is defined as the time interval **from k to k+1 seconds**.

System Constraints:

- **Chunk Duration:** 1.0 second.
- **Video Resolution:** 256x256
- **Audio Sample Rate:** 16,000 Hz (Fixed).
- **Audio Chunk Size:** 16,000 samples.
- **Context Scope:** Intra-chunk only. No temporal dependency exists across chunk boundaries.

1.2 Audio Segmentation (Deterministic)

Audio is treated as a contiguous stream of samples sliced by exact array indices to prevent drift.

- **Logic:** audio_chunk_k = Audio[k * 16000 : (k+1) * 16000].

- **Properties:** This method is drift-free (integer-based indexing) and fixed-length (exactly 16,000 samples).

1.3 Video Segmentation (Timestamp-Based)

Video segmentation is framerate-agnostic and driven by the Presentation Timestamp (PTS).

- **Logic:** Given a frame F_i with timestamp t_i (in seconds), the Chunk ID is calculated as the integer floor of the timestamp: chunk_id(F_i) = floor(t_i).

- **Properties:**
  - **Variable Count:** The number of frames per chunk may vary (e.g., 29 frames in Chunk 0, 30 frames in Chunk 1).

  - **Alignment:** Frame F_i belongs to Chunk k if and only if it occurs during second k.

1.4 Global Indexing & Iteration

The system utilizes a unified, zero-based integer index **k** to manage processing across both modalities.

- **Index Definition:** The index `k` corresponds directly to the timeline in seconds. For example, `k=0` refers to the interval `[0.0, 1.0)`, and `k=5` refers to `[5.0, 6.0)`.
- **Universal Retrieval:** A single value of `k` is used to retrieve the corresponding data pair:
  - **Audio:** Accessed via explicit array slicing `[k*16000 : (k+1)*16000]`.
  - **Video:** Accessed by filtering frames where `floor(timestamp) == k`.
- **Sequential Processing:** The encoder and decoder loops strictly iterate through `k = 0, 1, 2, ...` up to the total duration of the media, ensuring that the `k-th` audio chunk is always processed alongside the `k-th` video chunk.

Part 2: The Audio Subsystem (WavinWav Implementation)

The audio module acts as the "Forward Concealing Process" using an Invertible Neural Network (INN) to embed secret audio into cover audio.

2.1 Audio Preprocessing

1. **Resampling:** Both cover (x_cover) and secret (x_secret) signals are resampled to 16 kHz.

2. **Segmentation:** Audio is divided into the fixed 1-second (16,000 sample) segments defined in Part 1.

3. **Normalization:** Chunks are normalized and prepared as input pairs.

2.2 Forward Pass (Invertible Blocks)

The paired segments are fed into a sequence of **N** invertible blocks. Each block performs the following:

- **Coupling Layers:** Inputs are split. One half remains unchanged and generates parameters (scaling and shifting) via a convolutional network to transform the second half.

- **Channel Shuffling:** Data is shuffled between blocks to ensure all samples are processed.

The Affine Transformation (Math)

The transformation is designed for mathematical reversibility using scaling (**s**) and shifting (**t**) parameters.

- **Scaling (s):** Multiplies the input. The network computes this to "compress" the secret audio into unnoticeable variations of the cover. It is applied via an exponential function to ensure positivity: x * exp(s).

- **Shifting (t):** Adds an offset to "slide" the secret data into a specific range within the cover signal.

Forward Equation: y2 = x2 * exp(s) + t Where y1 remains identical to x1.

Reverse Equation (Extraction): x2 = (y2 - t) * exp(-s) Because the decoder has access to y1 (which equates to x1), it can regenerate the exact **s** and **t** values to reverse the math.

2.3 Outputs & Loss

- **Outputs:**
    - **Stego Audio (x_stego):** The primary output containing the hidden data.

    - **Lost Variable (z_lost):** An auxiliary variable capturing information that did not fit

into the stego audio. During training, this is forced to follow a Gaussian distribution N(0, 1).

- **Optimization:** A **Time-Frequency Loss** is applied, calculating differences in both the time domain (raw samples) and frequency domain (STFT) to minimize audible artifacts.

Part 3: The Video Subsystem (V-INN Implementation)

This module is a reusable "Reversible Video Hiding Core" inspired by LF-VSN but adapted for single-secret embedding with a redundancy predictor.

3.1 Data Flow & Windowing

- **Sliding Window:** The system uses a sliding window of length **L** (e.g., L=3). For timestep **t**, it forms groups of cover and secret frames representing the range [t - (L//2) : t + (L//2) + 1].

- **Center Frame Output:** The encoder outputs only the central stego frame stego[t].

- **Decoder "Broadcast Trick":** During decoding, the single stego[t] frame is broadcast into a pseudo-group to match the input shape. A specific enhancement adds a tiny, deterministic sinusoidal positional code to each slot in the pseudo-group to provide structured input.

3.2 Transform Fusion Module

To avoid concatenation artifacts, the system uses a swappable "TransformFusion" module:

- **Option A (Fixed):** Haar wavelet transform with a learnable 1x1 convolution mixer for bands.

- **Option B (Learnable):** A learnable lifting scheme or pseudo-wavelet via strided convolutions.

3.3 Invertible Core Architecture

The core uses a two-stream reversible coupling block:

1. **Inputs:** Cover features (Ck) and Secret features (Sk).

2. **Forward Logic:**

- Ck+1 = Ck + g_s(Sk) (Additive coupling).
- Sk+1 = Sk * exp(clamp(g_c_scale(Ck+1))) + g_c_shift(Ck+1) (Affine coupling).

3. **Novelty:** Uses depthwise separable convolutions or tiny 3D convs inside the subnets (g) and invertible 1x1 permutations between blocks.

3.4 Redundancy Estimator

Instead of sampling, the system predicts redundancy (Z) from the stego group.

- **Architecture:** A small residual CNN (no Batch Norm) that estimates z_hat from the stego context.
- **Role:** This estimated z_hat is fed into the reverse pass of the Invertible Core to reconstruct the secret.

3.5 Training Losses (Single-Secret)

The total loss is L = Lf + lambda * Lb (with lambda approx. 2-6).

- **Forward Loss (Lf):** ||stego[t] - cover[t]||_1 + gamma * (1 - MS-SSIM).
- **Backward Loss (Lb):** ||secret_rec - secret||_1 + ||cover_rec - cover||_1.
- **Regularizers:** Frequency-domain penalties on high bands and stego distribution matching.

Note - The Structural Similarity Index Measure (SSIM) is a metric used to measure the similarity between two images (a reference and a processed image) by evaluating changes in luminance, contrast, and structure, rather than just pixel-by-pixel differences

---

Part 4: Combined Processing Pipeline

The encoder processes Cover (C) and Secret (S) signals in parallel 1-second chunks, integrating the subsystems defined above.

4.1 Encoder Implementation

The process iterates through synchronized chunks **k**:

1. **Audio Processing:**

    - Slice exact sample ranges: k * 16000 to (k+1) * 16000.
    - Execute AudioINN.forward (See Section 2.2).
2. **Video Processing:**
    - Retrieve frames where floor(timestamp) == k.
    - **Boundary Handling:** To maintain the "No Cross-Chunk Context" rule, generate temporal windows using **Replication Padding**. If a window requires frames from chunk k-1, replicate the first frame of chunk k. If it requires frames from k+1, replicate the last frame of chunk k.
    - Execute VideoINN.forward (See Section 3.3) on the window to generate the stego frame.

4.2 Decoder Implementation

The decoder mirrors the slicing logic exactly to ensure reversibility.

1. **Audio:** Slices stego audio into 16,000-sample chunks -> AudioINN.reverse().
2. **Video:** Groups stego frames by timestamp floor -> Rebuilds intra-chunk windows (using the Broadcast Trick) -> Predicts Redundancy (z_hat) -> VideoINN.reverse().

4.3 Design Robustness

- **Drift-Free:** Audio uses integer-based indexing; no floating-point accumulation errors occur.
- **VFR Resilience:** Video chunking relies on individual frame timestamps, making it robust to Variable Frame Rate (VFR) or frame drops.
- **Independence:** All embedding/extraction is performed independently per chunk.